Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота № 2

з дисципліни «Протоколи та алгоритми електронного голосування»

Виконав: студент групи ІП-34 Туболев М. Р.

Перевірив: Асистент кафедри, PhD Нестерук А. О.

2026

1. Тема, мета та завдання роботи

Тема: протокол Е-голосування зі сліпими підписами.

Мета роботи: програмно реалізувати всі кроки протоколу електронного голосування зі сліпими підписами для двох кандидатів і п’яти виборців, використовуючи алгоритм RSA як для сліпого електронного цифрового підпису (ЕЦП), так і для шифрування бюлетенів; дослідити, як сліпий підпис відокремлює перевірку права виборця від подання голосу та як схема вибіркового розкриття «9 з 10» контролює коректність бюлетенів.

Завдання роботи:

- реалізувати формування виборчою комісією (ВК) списку виборців і кандидатів, генерацію ВК пари ключів RSA та публікацію відкритого ключа;
- реалізувати формування кожним виборцем 10 наборів бюлетенів (по одному бюлетеню на кандидата) з окремим випадковим ID для кожного набору;
- реалізувати обчислення хешів бюлетенів, їх маскування випадковими множниками та передачу до ВК 10 наборів замаскованих хешів;
- реалізувати перевірку ВК особи виборця, його наявності у списку та факту, що він раніше не отримував підписаний набір;
- реалізувати випадковий вибір ВК 9 із 10 наборів, їх розкриття виборцем і перевірку ВК правильності формування та відповідності замаскованим хешам;
- реалізувати сліпий підпис ВК десятого, нерозкритого набору та зняття виборцем маскування з отриманих підписів;
- реалізувати вибір бюлетеня, додавання підпису ВК, шифрування відкритим ключем ВК, розшифрування, перевірку ЕЦП і унікальності ID, прийняття або відхилення бюлетеня;
- реалізувати підрахунок голосів і публікацію результатів разом зі списком ID і підписаних бюлетенів;
- перевірити неможливість повторного отримання підписаного набору, відхилення другого бюлетеня з того самого набору та виявлення некоректного набору серед розкритих;

– для кожної перевірки показати як позитивний, так і негативний сценарій.

2. Короткий опис програмної реалізації

Протокол реалізовано мовою TypeScript і виконується середовищем Node.js без етапу збірки. Криптографічні бібліотеки не використовувалися: модульна арифметика, генерація простих чисел, розширений алгоритм Евкліда, блокове шифрування RSA, хеш квадратичної згортки та сліпий підпис написані вручну, щоб формули збігалися з методичними вказівками. Великі числа представлені типом bigint, довжина модуля RSA виборчої комісії — 512 біт.

Модуль	Призначення
src/bigint-math.ts	модульне піднесення до степеня, розширений алгоритм Евкліда, обернене за модулем, тест Міллера — Рабіна, генерація простих чисел
src/rsa.ts	генерація пари ключів, блокове шифрування та розшифрування, хеш квадратичної згортки, перевірка ЕЦП
src/blind-signature.ts	сліпий підпис RSA: множник маскування, маскування хешу, сліпий підпис, зняття маскування
src/types.ts	бюлетень, набори, запит на підпис, розкриття, конверт, причини відмов, строга серіалізація бюлетеня
src/voter.ts	виборець: 10 наборів бюлетенів, маскування, розкриття, зняття маскування, анонімне голосування, перевірка свого голосу
src/commission.ts	ВК: списки, перевірка особи, вибір 9 з 10, перевірка наборів, сліпий підпис, приймання бюлетенів, підрахунок
src/main.ts	демонстраційний сценарій: позитивні та негативні випадки, статистичний експеримент
tests/	29 модульних тестів (вбудований тест-ранер node:test)

Таблиця 1 — Структура програмної реалізації

2.1. Ключі RSA виборчої комісії

ВК обирає два різні прості числа p і q по 256 біт, обчислює $n = p \cdot q$ і $\varphi(n) = (p-1)(q-1)$, відкритий показник $e = 65537$ (взаємно простий з $\varphi(n)$) і закритий показник $d = e^{-1} \bmod \varphi(n)$, знайдений розширеним алгоритмом Евкліда. Відкритий ключ (e, n) публікується: ним виборці маскують хеші, перевіряють підпис ВК і шифрують бюлетені.

```
/**
 * Генерація пари ключів RSA.
 * p, q — різні прості; n = p*q; φ(n) = (p-1)(q-1);
 * e — непарне, 1 < e < φ(n), НСД(e, φ(n)) = 1; d = e⁻¹ mod φ(n).
 */
export function generateKeyPair(bits = 512): KeyPair {
  if (bits < 64) throw new RangeError("Для блокового шифрування потрібно
щонайменше 64 біти");

  const halfBits = Math.floor(bits / 2);
  let p = generatePrime(halfBits);
  let q = generatePrime(bits - halfBits);
  while (p === q) q = generatePrime(bits - halfBits);

  const n = p * q;
  const phi = (p - 1n) * (q - 1n);

  const e = gcd(DEFAULT_PUBLIC_EXPONENT, phi) === 1n
    ? DEFAULT_PUBLIC_EXPONENT
    : findCoprimeExponent(phi);
  const d = modInverse(e, phi); // розширений алгоритм Евкліда

  return { publicKey: { e, n }, privateKey: { d, n }, parameters: { p, q,
phi } };
}
```

Лістинг 1 — Генерація пари ключів (src/rsa.ts)

2.2. Сліпий підпис RSA

Бюлетень хешується функцією квадратичної згортки $H_i = (H_{i-1} + M_i)^2 \bmod n$. Виборець обирає випадковий множник r, взаємно простий з n, і маскує хеш: $H' = H \cdot r^e \bmod n$. ВК підписує замаскований хеш, не бачачи H: $S' = (H')^d \bmod n$. Виборець знімає маскування: $S = S' \cdot r^{-1} \bmod n$, де r^{-1} знаходиться розширеним алгоритмом Евкліда. Оскільки $(H \cdot r^e)^d = H^d \cdot r \bmod n$, отримане S дорівнює звичайному підпису $H^d \bmod n$. Перевірка: $H_c = S^e \bmod n$ має дорівнювати H.

```
/** Випадковий множник маскування r: 1 < r < n, НСД(r, n) = 1. */
export function randomBlindingFactor(n: bigint): bigint {
  while (true) {
    const r = randomBigInt(2n, n - 2n);
    if (gcd(r, n) === 1n) return r;
  }
}

export function blind(hash: bigint, r: bigint, key: PublicKey): bigint {
  return (hash * modPow(r, key.e, key.n)) % key.n;
}

export function signBlinded(blinded: bigint, key: PrivateKey): bigint {
  return modPow(blinded, key.d, key.n);
}

/** r⁻¹ знаходиться розширеним алгоритмом Евкліда. */
export function unblind(blindSignature: bigint, r: bigint, key: PublicKey):
bigint {
  return (blindSignature * modInverse(r, key.n)) % key.n;
}

/** Перевірка ЕЦП: порівнюємо хеш повідомлення H і H_c = S^e mod n. */
export function verify(message: string, signature: bigint, key: PublicKey):
boolean {
  return hashMessage(message, key.n) === modPow(signature, key.e, key.n);
}
```

Лістинг 2 — Маскування, сліпий підпис, зняття маскування та перевірка ЕЦП (src/blind-signature.ts, src/rsa.ts)

2.3. Формування наборів бюлетенів виборцем

Виборець формує 10 наборів; у кожному — по бюлетеню для кожного кандидата. Бюлетень містить лише випадковий 64-бітний ID набору та код кандидата — жодних персональних даних. ID усіх наборів різні. Для кожного бюлетеня обчислюється хеш і власний множник маскування, які виборець зберігає у себе; до ВК надходять лише замасковані хеші. Параметр choicesFor використовується тільки в негативних сценаріях для формування шахрайського набору.

```
/**
 * Кроки 2–4: 10 наборів по бюлетеню на кандидата, окремий випадковий ID на
набір,
 * хеш кожного бюлетеня маскується власним множником r.
 * `choicesFor` дозволяє сформувати набір з іншими бюлетенями (сценарій
шахрайства).
 */
prepare(
  candidates: readonly Candidate[],
  commissionKey: PublicKey,
  choicesFor: (setIndex: number) => readonly string[] = () =>
candidates.map((candidate) => candidate.id),
): SigningRequest {
  const ids = new Set<string>();
  while (ids.size < SET_COUNT) ids.add(randomBytes(8).toString("hex"));

  this.sets = [...ids].map((id, index) => {
    const ballots = choicesFor(index).map((candidateId) =>
serializeBallot({ id, candidateId }));
    const hashes = ballots.map((ballot) => hashMessage(ballot,
commissionKey.n));
    const factors = ballots.map(() =>
randomBlindingFactor(commissionKey.n));
    const blinded = hashes.map((hash, j) => blind(hash, factors[j]!,
commissionKey));
    return { id, ballots, hashes, factors, blinded };
  });

  return {
    voterName: this.name,
    document: this.document,
    blindedSets: this.sets.map((set) => set.blinded),
  };
}
```

Лістинг 3 — Формування 10 наборів і маскування хешів (src/voter.ts)

2.4. Перевірка особи та вибір наборів для розкриття

Отримавши запит, ВК перевіряє наявність виборця у списку, відповідність пред’явленого документа, відсутність позначки про видачу підписаного набору (або попередньої відмови) та структуру запиту: рівно 10 наборів по одному замаскованому хешу на кандидата. Далі ВК криптографічно стійким генератором обирає один набір, який залишиться нерозкритим, і повідомляє виборцю номери інших дев’яти.

```
/** Кроки 5–6: перевірка особи та права на підписаний набір, вибір 9
наборів для розкриття. */
requestSigning(request: SigningRequest): ChallengeResult {
  const voter = this.voters.get(request.voterName);
  if (!voter) {
    this.note("inList", false, `«${request.voterName}» відсутній у списку
виборців`);
    return rejected("NOT_IN_LIST", SIGNING_REJECTIONS);
  }
  this.note("inList", true, `«${voter.name}» є у списку виборців`);

  if (voter.document !== request.document) {
    this.note("identity", false, `документ ${request.document} не належить
«${voter.name}»`);
    return rejected("IDENTITY_MISMATCH", SIGNING_REJECTIONS);
  }
  this.note("identity", true, `особу підтверджено документом
${voter.document}`);

  if (voter.status !== "registered") {
    const reason = voter.status === "issued" ? "ALREADY_ISSUED" :
"REFUSED_EARLIER";
    this.note("notIssued", false, SIGNING_REJECTIONS[reason]);
    return rejected(reason, SIGNING_REJECTIONS);
  }
  this.note("notIssued", true, "у реєстрі ВК немає позначки про видачу");

  const wellFormed =
    request.blindedSets.length === SET_COUNT &&
    request.blindedSets.every((set) => set.length ===
this.candidates.length);
  this.note(
    "requestShape",
    wellFormed,
    `${request.blindedSets.length} наборів ×
${request.blindedSets.map((set) => set.length).join("/")} замаск. хешів`,
  );
  if (!wellFormed) return rejected("MALFORMED_REQUEST",
SIGNING_REJECTIONS);

  const unopened = this.pickUnopened();
  const reveal = [...Array(SET_COUNT).keys()].filter((index) => index !==
unopened);
  this.pending.set(voter.name, { blindedSets: request.blindedSets, unopened
});
  this.note("challenge", true, `розкрити набори №${reveal.map((index) =>
index + 1).join(", ")}`);
  return { status: "challenge", reveal };
}
```

Лістинг 4 — Обробка запиту на сліпий підпис (src/commission.ts)

2.5. Розкриття, перевірка наборів і сліпий підпис

Виборець передає початкові бюлетені та множники маскування лише для обраних дев’яти наборів. ВК перевіряє, що розкрито саме ці набори, а в кожному з них: кількість бюлетенів дорівнює кількості кандидатів, бюлетені

не містять зайвих полів, мають однаковий ID, який не повторюється в інших наборах, кожен кандидат представлений рівно одним бюлетенем, а самостійно обчислене $H \cdot r^e \bmod n$ збігається з отриманим раніше замаскованим хешем. Якщо хоча б один набір некоректний, ВК відмовляє в підписі та фіксує відмову в реєстрі — інакше шахрай міг би повторювати спроби, доки некоректний набір не залишиться нерозкритим. Якщо всі дев’ять наборів правильні, ВК підписує кожен замаскований хеш десятого набору і робить позначку про видачу. ID десятого набору ВК не знає.

```
/** Кроки 8–9: перевірка 9 розкритих наборів і сліпий підпис десятого. */
completeSigning(voterName: string, revealed: readonly RevealedSet[]):
SigningResult {
  const voter = this.voters.get(voterName);
  const request = this.pending.get(voterName);
  if (!voter || !request) return rejected("NO_PENDING_REQUEST",
SIGNING_REJECTIONS);
  this.pending.delete(voterName);

  const expected = [...Array(SET_COUNT).keys()].filter((index) => index !==
request.unopened);
  const indices = revealed.map((set) => set.index).sort((a, b) => a - b);
  const indicesMatch = indices.join() === expected.join();
  this.note("revealedIndices", indicesMatch, `розкрито набори №
${indices.map((index) => index + 1).join(", ")}`);
  if (!indicesMatch) return this.refuse(voter, "WRONG_REVEAL");

  for (const set of revealed) {
    const problem = this.inspectSet(set, request.blindedSets[set.index] ??
[]);
    this.note("setValid", problem === undefined, `набір №${set.index + 1}:
${problem ?? "коректний"}`);
    if (problem !== undefined) return this.refuse(voter, "BAD_SET", `набір
№${set.index + 1}: ${problem}`);
  }

  const blindSignatures = (request.blindedSets[request.unopened] ??
[]).map((hash) =>
    signBlinded(hash, this.keys.privateKey),
  );
  voter.status = "issued";
  this.note("blindSign", true, `набір №${request.unopened + 1} підписано
наосліп: S' = (H')ᵈ mod n`);
  return { status: "signed", blindSignatures };
}

/** Повертає опис порушення або undefined, якщо набір коректний. */
private inspectSet(set: RevealedSet, blinded: BlindedSet): string |
undefined {
  if (set.ballots.length !== this.candidates.length || set.factors.length
!== set.ballots.length) {
    return `очікувалось ${this.candidates.length} бюлетені з множниками`;
  }

  let ballots: Ballot[];
  try {
    ballots = set.ballots.map(parseBallot);
  } catch {
    return "бюлетень містить зайві або некоректні дані";
  }

  const ids = new Set(ballots.map((ballot) => ballot.id));
  if (ids.size !== 1) return "бюлетені набору мають різні ID";
  const [id] = ids;
  if (id === undefined || this.revealedIds.has(id)) return `ID ${id}
повторюється в іншому наборі`;
  this.revealedIds.add(id);

  const choices = ballots.map((ballot) => ballot.candidateId).sort();
  const expected = this.candidates.map((candidate) => candidate.id).sort();
```

```
  if (choices.join() !== expected.join()) {
    return `бюлетені ${ballots.map((ballot) => ballot.candidateId).join(",
")} замість ${expected.join(", ")}`;
  }

  for (const [j, text] of set.ballots.entries()) {
    const r = set.factors[j] ?? 0n;
    if (gcd(r, this.publicKey.n) !== 1n) return `множник r бюлетеня ${j +
1} не взаємно простий з n`;
    if (blind(hashMessage(text, this.publicKey.n), r, this.publicKey) !==
blinded[j]) {
      return `H·rᵉ mod n бюлетеня ${j + 1} ≠ отриманому раніше H'`;
    }
  }
  return undefined;
}
```

Лістинг 5 — Перевірка розкритих наборів і сліпий підпис десятого (src/commission.ts)

Виборець знімає маскування з отриманих сліпих підписів і одразу перевіряє, що $S^e \bmod n = H$ для кожного бюлетеня, тобто що ВК повернула дійсні підписи.

```
/** Крок 7: розкрити лише набори, обрані ВК; десятий залишається таємним.
*/
reveal(indices: readonly number[]): RevealedSet[] {
  this.unopened = this.sets.findIndex((_, index) =>
!indices.includes(index));
  return indices.map((index) => {
    const set = this.sets[index];
    if (!set) throw new Error(`Набору №${index + 1} не існує`);
    return { index, ballots: set.ballots, factors: set.factors };
  });
}

/** Крок 10: S = S'·r⁻¹ mod n і перевірка, що це справді підпис ВК під
нашим бюлетенем. */
acceptSignatures(blindSignatures: readonly bigint[], commissionKey:
PublicKey): void {
  const set = this.unopened === undefined ? undefined :
this.sets[this.unopened];
  if (!set) throw new Error("Немає нерозкритого набору");

  this.signedBallots = new Map(
    set.ballots.map((ballot, j) => {
      const signature = unblind(blindSignatures[j] ?? 0n, set.factors[j]!,
commissionKey);
      if (!verify(ballot, signature, commissionKey)) throw new Error("ВК
повернула недійсний підпис");
      return [parseBallot(ballot).candidateId, { ballot, signature }];
    }),
  );
}
```

Лістинг 6 — Розкриття наборів і зняття маскування (src/voter.ts)

2.6. Анонімне голосування та перевірки ВК

Виборець обирає з нерозкритого набору бюлетень за свого кандидата, додає до нього підпис ВК і шифрує повідомлення відкритим ключем ВК блоковим RSA ($c = m^e \bmod n$). Конверт не містить ПІБ чи інших даних про відправника.

```
/** Кроки 11–12: обрати бюлетень, додати підпис ВК, зашифрувати відкритим
ключем ВК. */
vote(candidateId: string, commissionKey: PublicKey): Envelope {
  const signed = this.signedBallots.get(candidateId);
  if (!signed) throw new Error(`Немає підписаного бюлетеня за
«${candidateId}»`);
  this.castBallot = signed;
  return { cipher: sealFor(signed, commissionKey) };
}

/**
 * Шифрування бюлетеня з підписом відкритим ключем ВК.
 * Відкритий ключ доступний будь-кому — цим користуються негативні
сценарії.
 */
export function sealFor(signed: SignedBallot, commissionKey: PublicKey):
bigint[] {
  const payload = JSON.stringify({ ballot: signed.ballot, signature:
signed.signature.toString(16) });
  return encrypt(payload, commissionKey);
}
```

Лістинг 7 — Формування та шифрування повідомлення з бюлетенем (src/voter.ts)

ВК розшифровує повідомлення закритим ключем ($m = c^d \bmod n$), обчислює хеш бюлетеня та перевіряє власний ЕЦП, перевіряє існування кандидата та унікальність ID. Бюлетень приймається лише тоді, коли всі перевірки пройдено; бюлетень з уже використаним ID відхиляється, тому з одного підписаного набору зараховується лише перший бюлетень.

```
/** Кроки 12–13: анонімний бюлетень — розшифрування, перевірка ЕЦП ВК та
унікальності ID. */
receive(envelope: Envelope): VoteResult {
  const opened = this.open(envelope);
  if (!opened.ok) {
    this.note("decrypt", false, opened.error);
    return this.reject("UNDECRYPTABLE", opened.error);
  }
  const { text, ballot, signature } = opened;
  this.note("decrypt", true, `бюлетень: ${text}`);

  const signatureValid = verify(text, signature, this.publicKey);
  this.note("signature", signatureValid, signatureValid ? "H = H_c, підпис
ВК справжній" : "H ≠ H_c, підпис недійсний");
  if (!signatureValid) return this.reject("BAD_SIGNATURE");

  const candidate = this.candidates.find((item) => item.id ===
ballot.candidateId);
  this.note("ballotValid", candidate !== undefined, candidate ? `голос за
«${candidate.name}»` : `кандидата «${ballot.candidateId}» немає`);
  if (!candidate) return this.reject("INVALID_BALLOT");

  if (this.accepted.has(ballot.id)) {
    this.note("uniqueId", false, `ID ${ballot.id} уже використано`);
    return this.reject("DUPLICATE_ID", `ID ${ballot.id}`);
  }
  this.note("uniqueId", true, `ID ${ballot.id} використовується вперше`);

  this.accepted.set(ballot.id, { ...ballot, ballot: text, signature });
  return { status: "accepted", ballot };
}
```

Лістинг 8 — Приймання бюлетеня виборчою комісією (src/commission.ts)

2.7. Підрахунок, публікація та перевірка виборцем

Після завершення голосування ВК підраховує голоси та публікує результати разом зі списком ID і відповідних їм підписаних бюлетенів. Кожен виборець знаходить у списку свій ID і порівнює опублікований бюлетень і підпис із тими, що надсилав: збіг означає, що голос враховано правильно, відсутність ID — що голос втрачено, розбіжність — що запис змінено.

```
/** Крок 14: підрахунок і публікація результатів разом з усіма ID та
підписаними бюлетенями. */
publishResults(): ElectionResults {
  const published = [...this.accepted.values()];
  const tally = new Map(
    this.candidates.map((candidate) => [
      candidate.id,
      published.filter((entry) => entry.candidateId ===
candidate.id).length,
    ]),
  );
  const max = Math.max(0, ...tally.values());
  const leaders = max === 0 ? [] : this.candidates.filter((candidate) =>
tally.get(candidate.id) === max);

  return {
    tally,
    published,
    accepted: published.length,
    rejected: this.rejectedCount,
    leaders,
    isTie: leaders.length > 1,
  };
}

/** Крок 14: знайти свій ID серед опублікованих бюлетенів і звірити вибір.
*/
checkPublished(published: readonly PublishedBallot[]): OwnVoteCheck {
  if (!this.castBallot) throw new Error("Виборець ще не голосував");
  const own = parseBallot(this.castBallot.ballot).id;
  const entry = published.find((item) => item.id === own);
  if (!entry) return "missing";
  return entry.ballot === this.castBallot.ballot && entry.signature ===
this.castBallot.signature
    ? "counted"
    : "altered";
}
```

Лістинг 9 — Публікація результатів і перевірка виборцем свого голосу (src/commission.ts, src/voter.ts)

3. Результати виконання протоколу та передбачених перевірок

Демонстраційний сценарій запускається командою npm start. У ньому беруть участь два кандидати та п’ять виборців; для кожної перевірки ВК передбачено як позитивний, так і негативний випадок. Зведення сценаріїв наведено в таблиці 2.

Сценарій	**Що перевіряється**	**Результат**
5 виборців отримують підписані набори	особа, список, структура запиту, 9 розкритих наборів	набори підписано наосліп, статус «набір видано»
5 бюлетенів подано анонімно	розшифрування, ЕЦП ВК, кандидат, унікальність ID	прийнято 5 бюлетенів, 3:2
1. Стороння особа	наявність у списку	відмова: NOT_IN_LIST
2. Чужий документ	перевірка особи	відмова: IDENTITY_MISMATCH
3. Повторний запит на набір	позначка про видачу	відмова: ALREADY_ISSUED
4. Два бюлетені з одного набору	унікальність ID	перший прийнято, другий: DUPLICATE_ID
5. Повторне надсилання перехопленого бюлетеня	унікальність ID	відхилено: DUPLICATE_ID
6. Зміна вибору в підписаному бюлетені	ЕЦП ВК: $H \neq H_c$	відхилено: BAD_SIGNATURE
7. Бюлетень без підпису ВК	ЕЦП ВК	відхилено: BAD_SIGNATURE
8. Шифрування не тим ключем	розшифрування	відхилено: UNDECRYPTABLE
9. ВК змінює опублікований запис	перевірка виборцем свого ID	підміну викрито
10. Набір з двома бюлетенями за К1	склад розкритого набору	відмова в підписі: BAD_SET
11. Підміна бюлетенів під час розкриття	$H \cdot r^e \bmod n = H'$	відмова в підписі: BAD_SET
12. Новий запит після відмови	реєстр ВК	відмова: REFUSED_EARLIER

Таблиця 2 — Перевірені сценарії роботи протоколу

На рисунку 1 показано перший крок: ВК сформувала список із п’яти виборців (ПІБ і документ), список двох кандидатів, згенерувала 512-бітну

пару ключів RSA та опублікувала відкритий ключ.

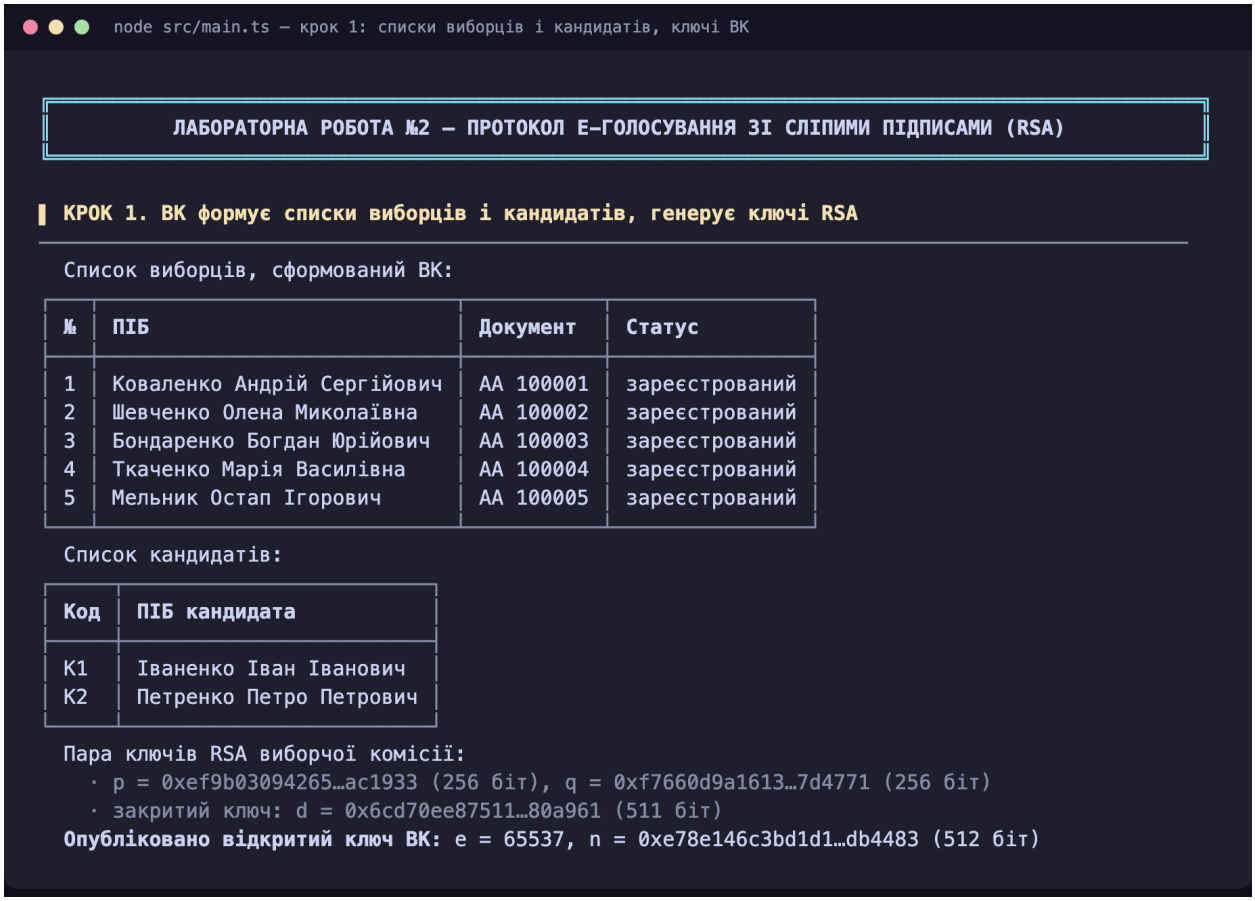


Рисунок 1 — Крок 1: списки виборців і кандидатів, ключі ВК

На рисунку 2 виборець № 1 сформував 10 наборів з різними випадковими ID і замаскував хеші всіх 20 бюлетенів. Для першого бюлетеня показано проміжні значення: текст, хеш H, множник r і замаскований хеш H'.

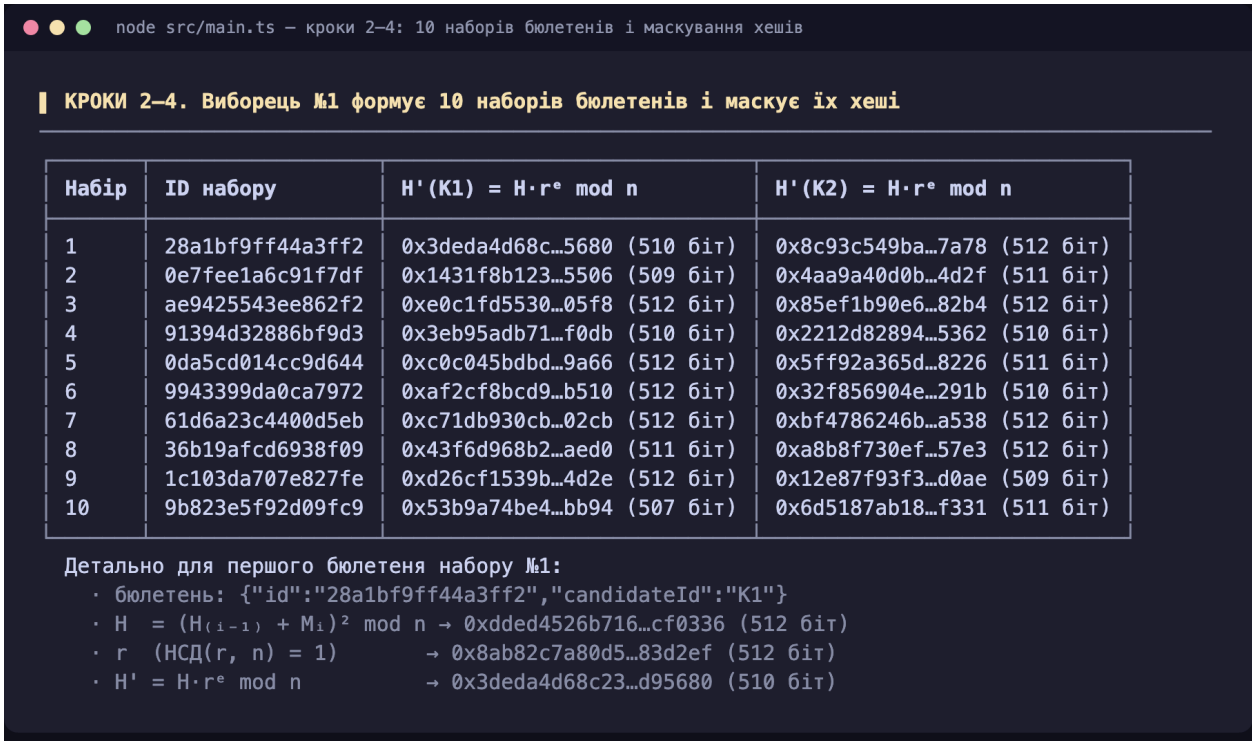


Рисунок 2 — Кроки 2–4: 10 наборів бюлетенів і маскування хешів

Рисунок 3 демонструє позитивний сценарій отримання підписаного набору: ВК перевірила особу та право на набір, випадково обрала 9 наборів, перевірила кожен з них, підписала наосліп десятий, а виборець зняв маскування й отримав дійсні підписи ВК на обидва бюлетені нерозкритого набору.

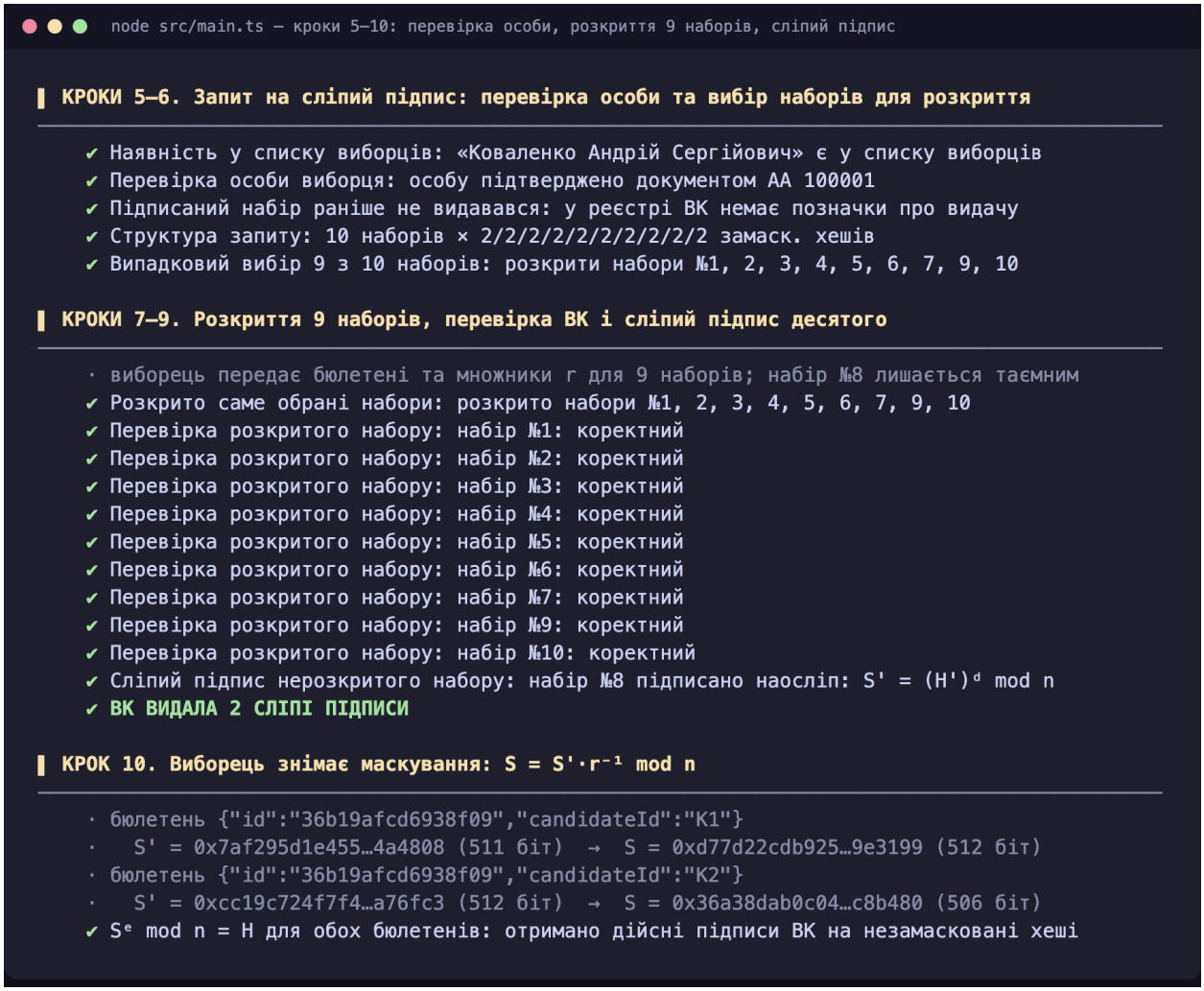


Рисунок 3 — Кроки 5–10: перевірка особи, розкриття 9 наборів, сліпий підпис і зняття маскування

На рисунку 4 решта виборців отримує підписані набори, після чого показано негативні сценарії етапу підписування: запит сторонньої особи, запит від імені виборця з чужим документом і повторний запит виборця, який уже отримав набір.

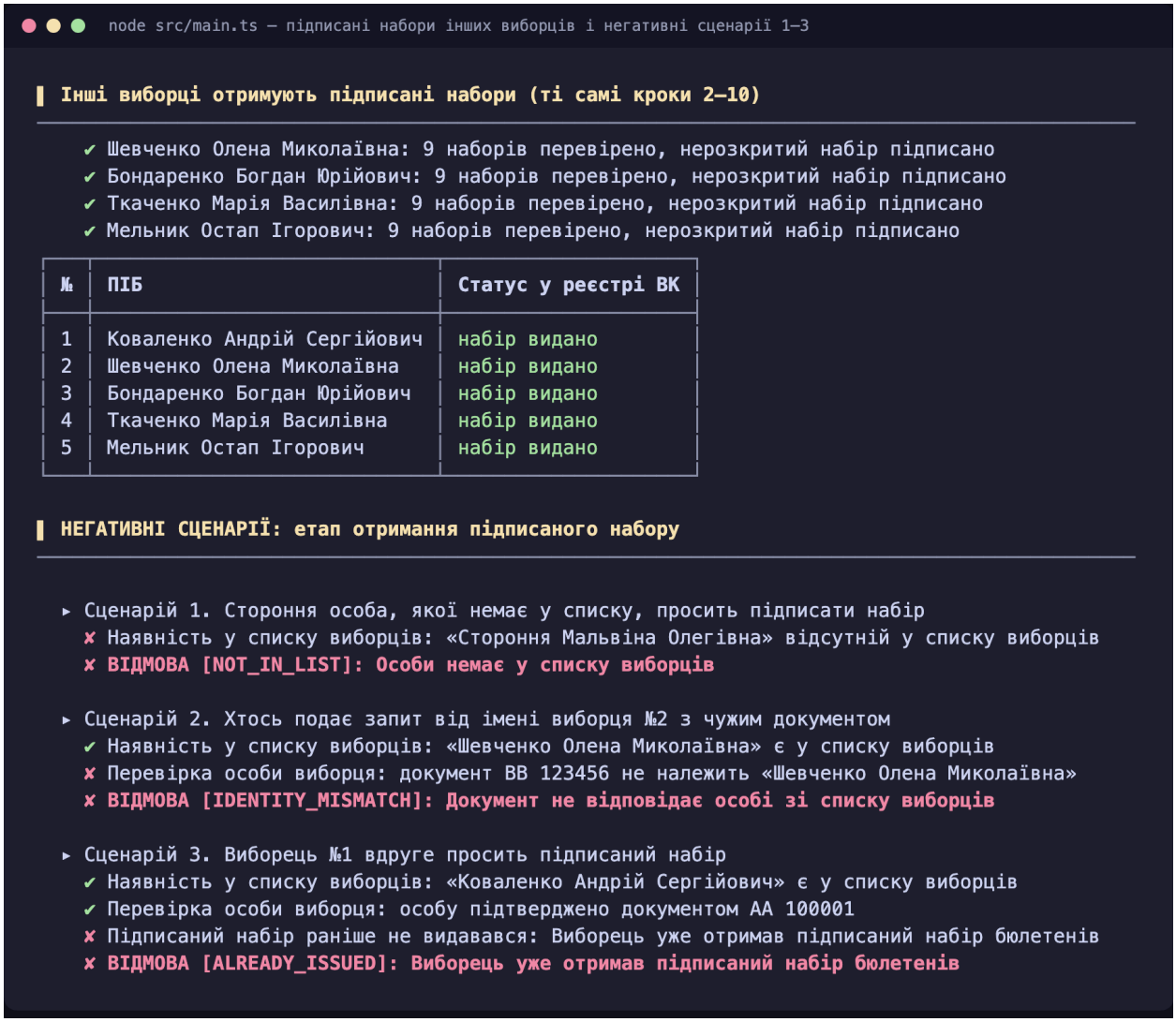


Рисунок 4 — Підписані набори інших виборців і негативні сценарії 1–3

Рисунок 5 демонструє анонімне голосування: кожен виборець надсилає один бюлетень з підписом ВК у зашифрованому вигляді без ПІБ, а ВК розшифровує його, перевіряє власний ЕЦП, кандидата й унікальність ID.

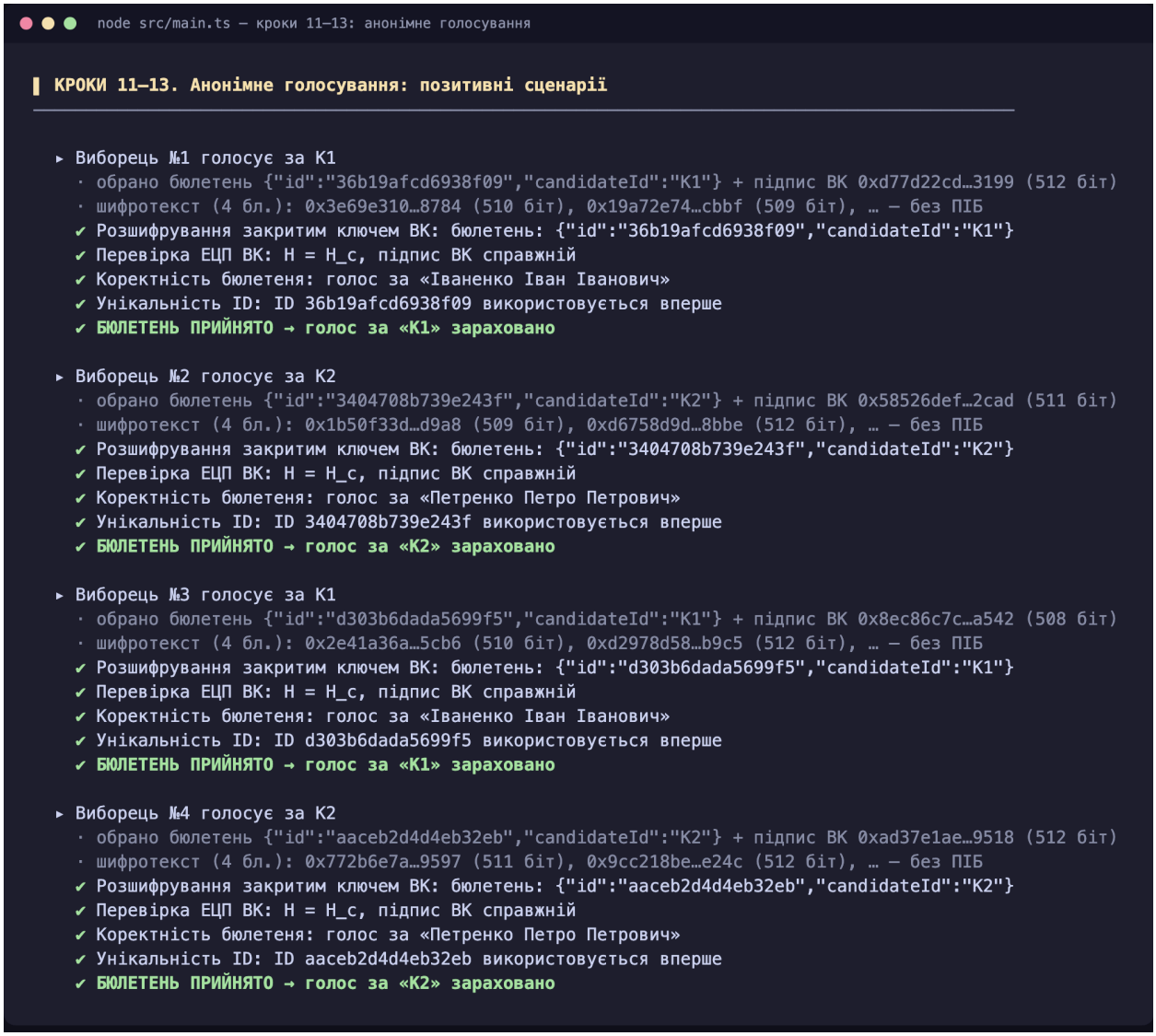


Рисунок 5 — Кроки 11–13: анонімне голосування та перевірки ВК

На рисунку 6 наведено негативні сценарії етапу голосування. Виборець № 5 надсилає обидва бюлетені свого набору: перший приймається, другий відхиляється через повторне використання ID. Повторне надсилання перехопленого бюлетеня також відхиляється за ID. Зміна вибору в підписаному бюлетені та бюлетень з довільним підписом не проходять перевірку ЕЦП, а повідомлення, зашифроване чужим ключем, неможливо розшифрувати.

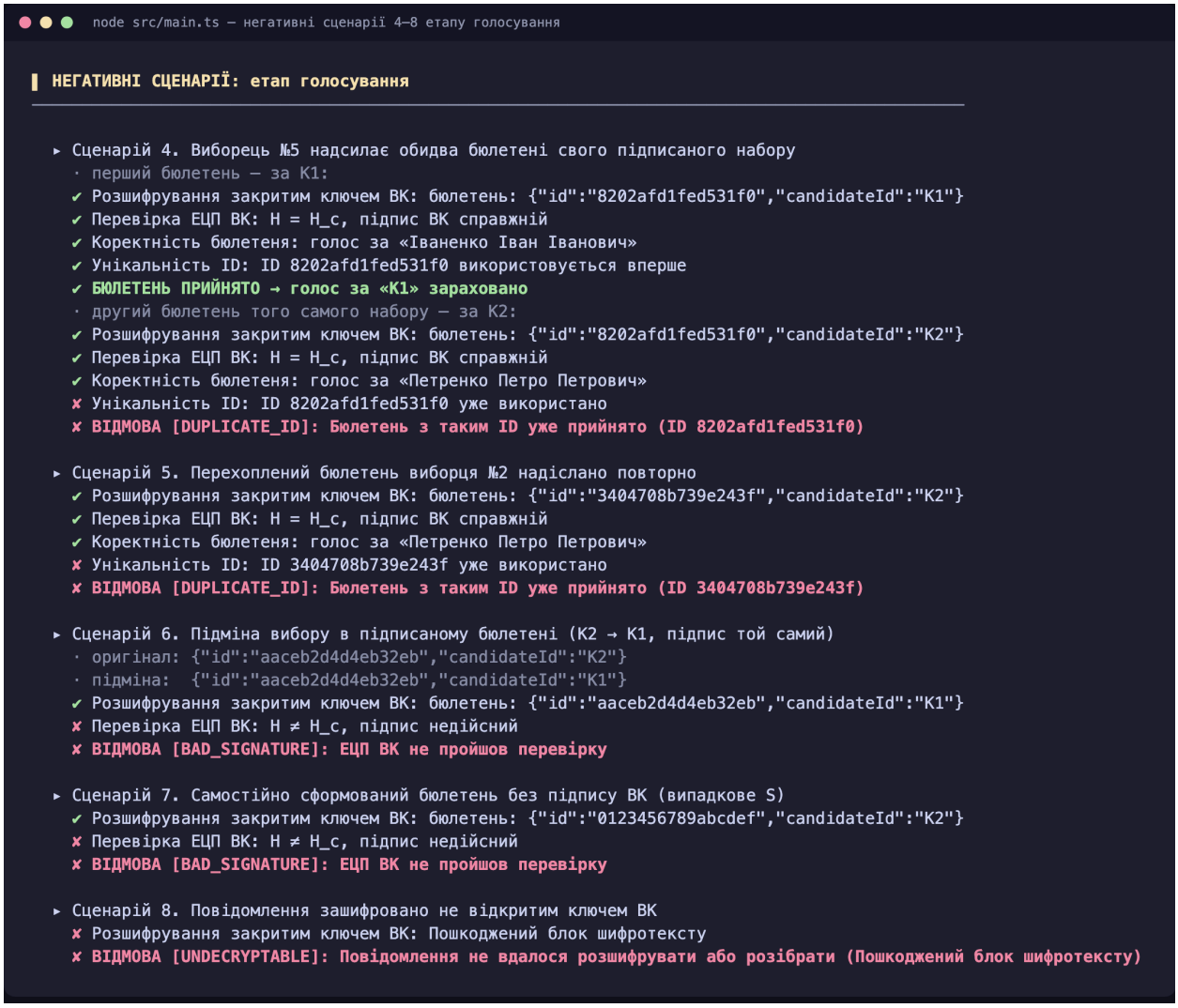


Рисунок 6 — Негативні сценарії етапу голосування

На рисунку 7 показано підрахунок голосів (3:2 на користь першого кандидата) і публікацію списку ID з підписаними бюлетенями. Кожен виборець знайшов свій ID і переконався, що голос враховано правильно. Жоден із поданих ID не входить до 45 ID розкритих наборів, відомих ВК, — тобто ВК не може пов’язати бюлетень з виборцем. Також показано, що підміну вибору в опублікованому записі виборець виявляє.

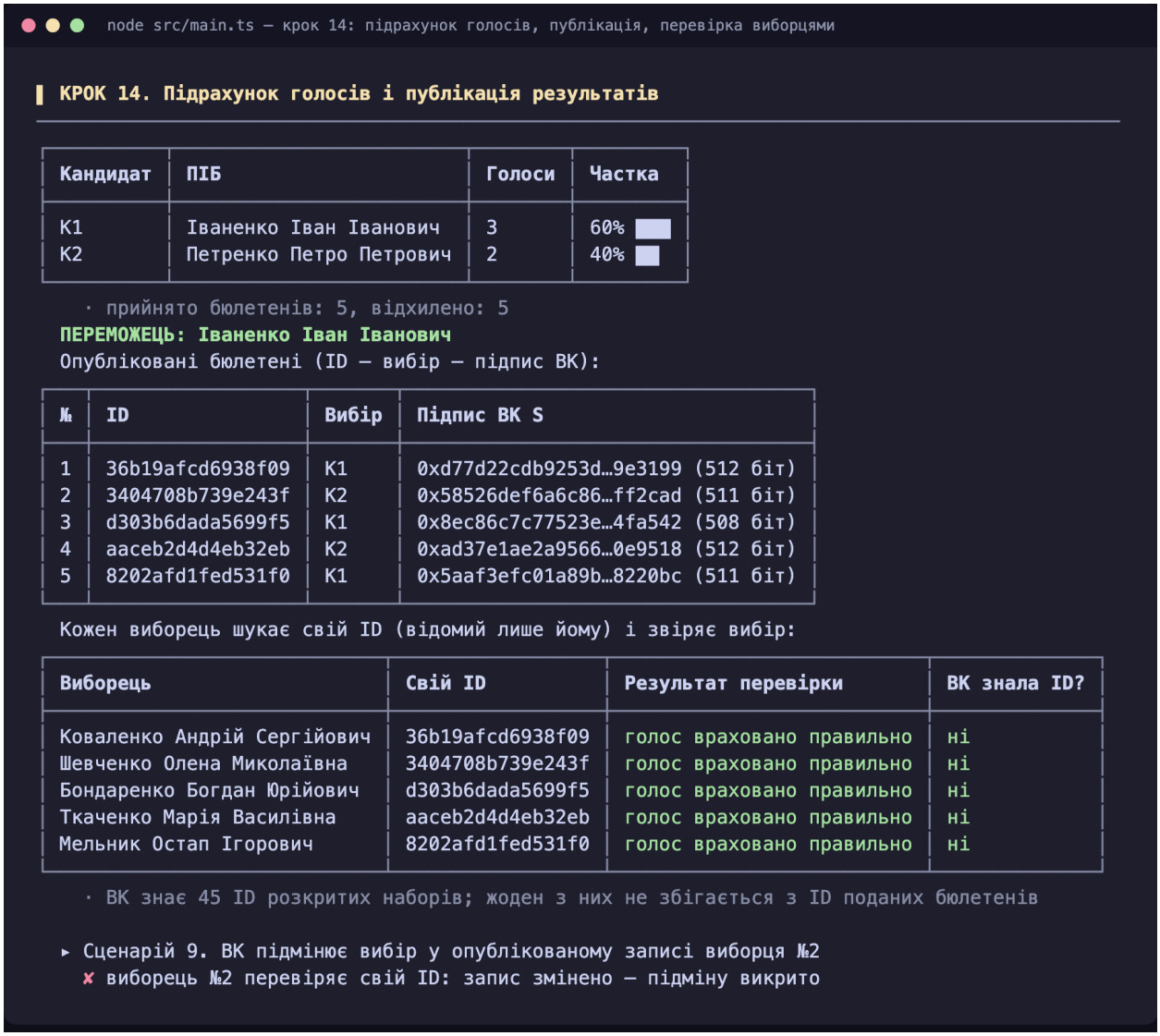


Рисунок 7 — Крок 14: підрахунок голосів, публікація результатів і перевірка виборцями

Рисунок 8 ілюструє виявлення некоректно сформованих наборів. У сценарії 10 набір №4 містить два бюлетені за К1; він потрапив до розкритих, тому ВК відмовила в підписі. У сценарії 11 бюлетені набору переставлено під час розкриття, і самостійно обчислене $H \cdot r^e \bmod n$ не збіглося з отриманим раніше H'. Після відмови повторний запит відхиляється. Статистичний експеримент з 1000 спроб підтверджує, що один некоректний набір виявляється приблизно у 90% випадків.

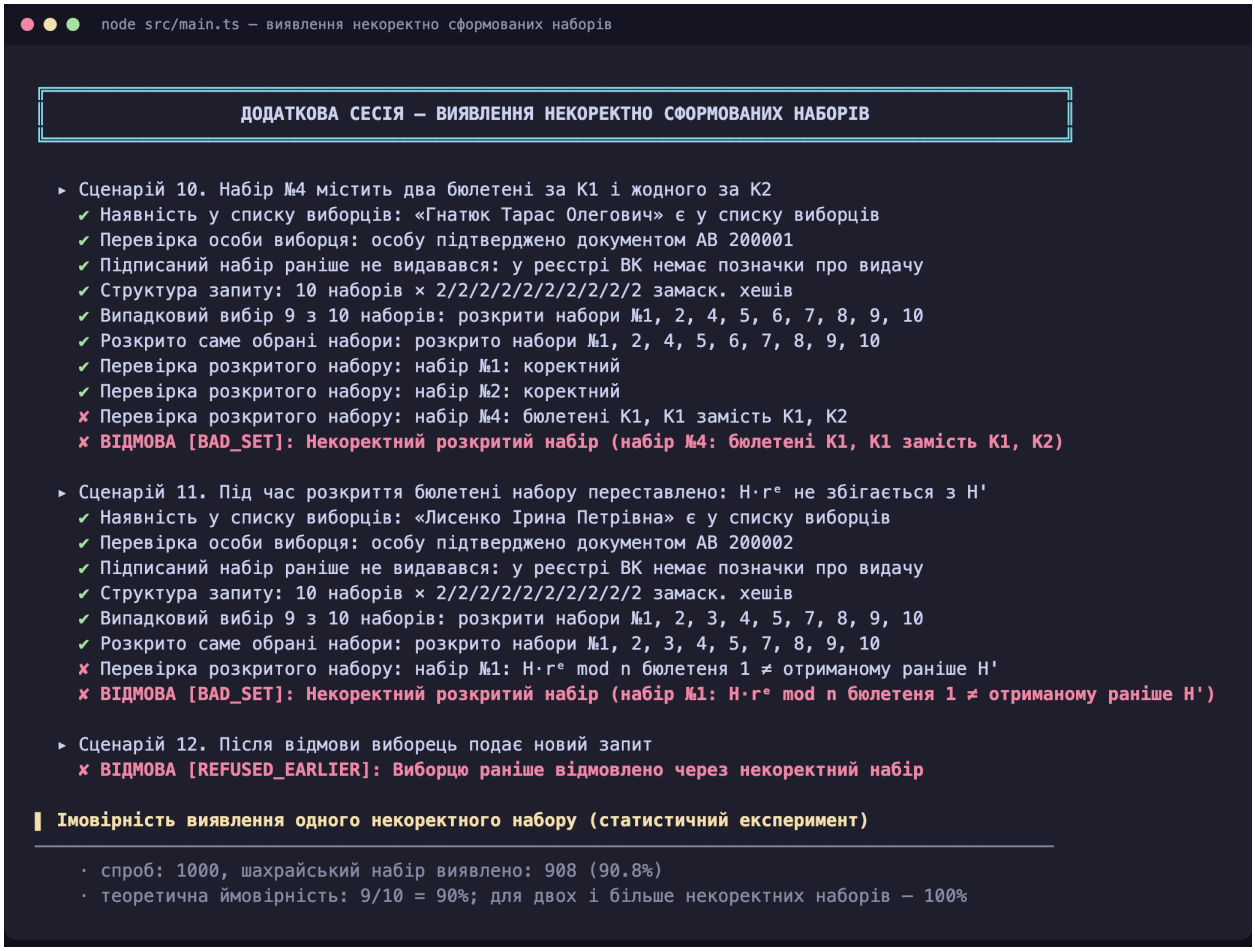


Рисунок 8 — Виявлення некоректно сформованих наборів і ймовірність виявлення

Зведення відповідності реалізованого протоколу вимогам до ідеального протоколу наведено на рисунку 9.

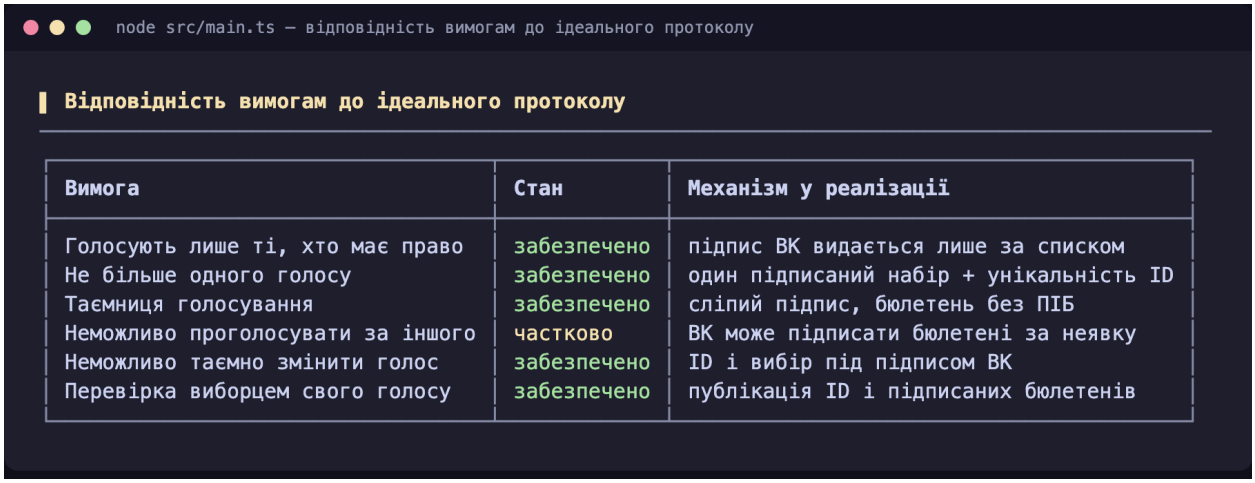


Рисунок 9 — Відповідність вимогам до ідеального протоколу

4. Тестування

Реалізацію покрито 29 модульними тестами (команда npm test). Тести перевіряють арифметику та RSA, властивості сліпого підпису (після зняття

маскування отримується звичайний підпис $H^d \bmod n$; без правильного r маскування не знімається; підпис не переноситься на змінене повідомлення), а також усі позитивні та негативні сценарії протоколу. Для детермінованості тести підставляють фіксований вибір нерозкритого набору; окремий тест показує, що некоректний набір, який залишився нерозкритим, проходить перевірку, — це підтверджує ймовірнісний характер схеми «9 з 10».

```
node --test tests/*.test.ts — модульні тести

▶ протокол зі сліпими підписами — позитивні сценарії
  ✔ ВК обирає 9 наборів для розкриття, десятий лишається таємним (8.24775ms)
  ✔ 5 виборців, 2 кандидати: голоси рахуються, кожен знаходить свій ID (23.336708ms)
  ✔ ВК не знає ID нерозкритого набору, яким виборець голосує (5.268458ms)
✔ протокол зі сліпими підписами — позитивні сценарії (37.609834ms)
▶ протокол зі сліпими підписами — негативні сценарії підписування
  ✔ особа поза списком не отримує підписаний набір (3.830958ms)
  ✔ запит з чужим документом відхиляється (4.541ms)
  ✔ повторно отримати підписаний набір неможливо (6.008208ms)
  ✔ запит не з 10 наборів відхиляється (3.713208ms)
  ✔ некоректний набір серед розкритих: відмова, повторний запит також відхиляється (5.4755ms)
  ✔ некоректний набір, що лишився нерозкритим, проходить — перевірка ймовірнісна (4.220958ms)
  ✔ розкритий бюлетень, що не відповідає замаскованому хешу, викривається (3.737625ms)
  ✔ розкриття не тих наборів відхиляється (3.04475ms)
✔ протокол зі сліпими підписами — негативні сценарії підписування (34.816584ms)
▶ протокол зі сліпими підписами — негативні сценарії голосування
  ✔ з одного підписаного набору приймається лише перший бюлетень (5.879708ms)
  ✔ зміна вибору в підписаному бюлетені виявляється перевіркою ЕЦП (6.170084ms)
  ✔ бюлетень без підпису ВК відхиляється (2.064917ms)
  ✔ повідомлення, зашифроване чужим ключем, не розшифровується (7.542083ms)
  ✔ виборець помічає зміну або зникнення свого бюлетеня в опублікованому списку (7.845167ms)
✔ протокол зі сліпими підписами — негативні сценарії голосування (29.609167ms)
▶ арифметика
  ✔ розширений алгоритм Евкліда дає коефіцієнти рівності a*x + b*y = НСД(a, b) (0.54925ms)
  ✔ обернене за модулем задовольняє (a * a⁻¹) mod m = 1 (0.09ms)
  ✔ modInverse відмовляє, якщо числа не взаємно прості (0.145209ms)
  ✔ тест простоти відрізняє прості від складених (0.39025ms)
  ✔ modPow збігається з наївним піднесенням до степеня (0.074958ms)
✔ арифметика (1.980041ms)
▶ RSA
  ✔ ключі узгоджені: e*d ≡ 1 mod φ(n) (3.558459ms)
  ✔ шифрування та розшифрування повертають вихідний текст (3.458083ms)
  ✔ хеш квадратичної згортки чутливий до зміни одного символу (4.352041ms)
✔ RSA (11.518167ms)
▶ сліпий підпис RSA
  ✔ множник маскування взаємно простий з n (0.28075ms)
  ✔ замаскований хеш не збігається з початковим (0.207542ms)
  ✔ після зняття маскування отримано дійсний підпис початкового повідомлення (0.536084ms)
  ✔ підпис не переноситься на змінене повідомлення (0.39225ms)
  ✔ без правильного r маскування не знімається (0.5025ms)
✔ сліпий підпис RSA (2.045709ms)
ℹ tests 29
ℹ pass 29
ℹ fail 0
ℹ duration_ms 315.484542
```

Рисунок 10 — Результати запуску модульних тестів

5. Висновок

У ході виконання лабораторної роботи програмно реалізовано всі кроки протоколу електронного голосування зі сліпими підписами для двох кандидатів і п’яти виборців: формування списків і ключів ВК, підготовку 10 наборів бюлетенів з випадковими ID, маскування хешів, перевірку особи виборця, випадковий вибір і перевірку 9 наборів, сліпий підпис десятого, зняття маскування, анонімне подання зашифрованого бюлетеня, перевірку ЕЦП і унікальності ID, підрахунок голосів та публікацію результатів зі списком ID.

Експериментально підтверджено, що сліпий підпис відокремлює перевірку права виборця від подання голосу: ВК перевіряє особу на етапі підписування, але не знає ID нерозкритого набору, а під час голосування бюлетень не містить персональних даних. Одноразовість голосування контролюється двічі — видачею одного підписаного набору та унікальністю ID; зміна вибору або ID руйнує підпис ВК; опублікований список дозволяє кожному виборцю перевірити врахування свого голосу.

Водночас протокол залишає значну довіру до ВК: володіючи закритим ключем, вона може сформувати додаткові бюлетені з новими ID, зокрема за виборців, які не проголосували, і окремий виборець цього не виявить. Перевірка наборів є ймовірнісною: один некоректний набір проходить з імовірністю 10%. Протокол також значно збільшує обсяг обчислень і даних (20 маскувань і 18 перевірок на одного виборця замість одного підпису). Крім того, в реалізації, як і в методичних вказівках, одна пара ключів ВК використовується і для підпису, і для шифрування; на практиці ці ключі слід розділяти, оскільки сліпий підпис еквівалентний розшифруванню довільного числа закритим ключем.